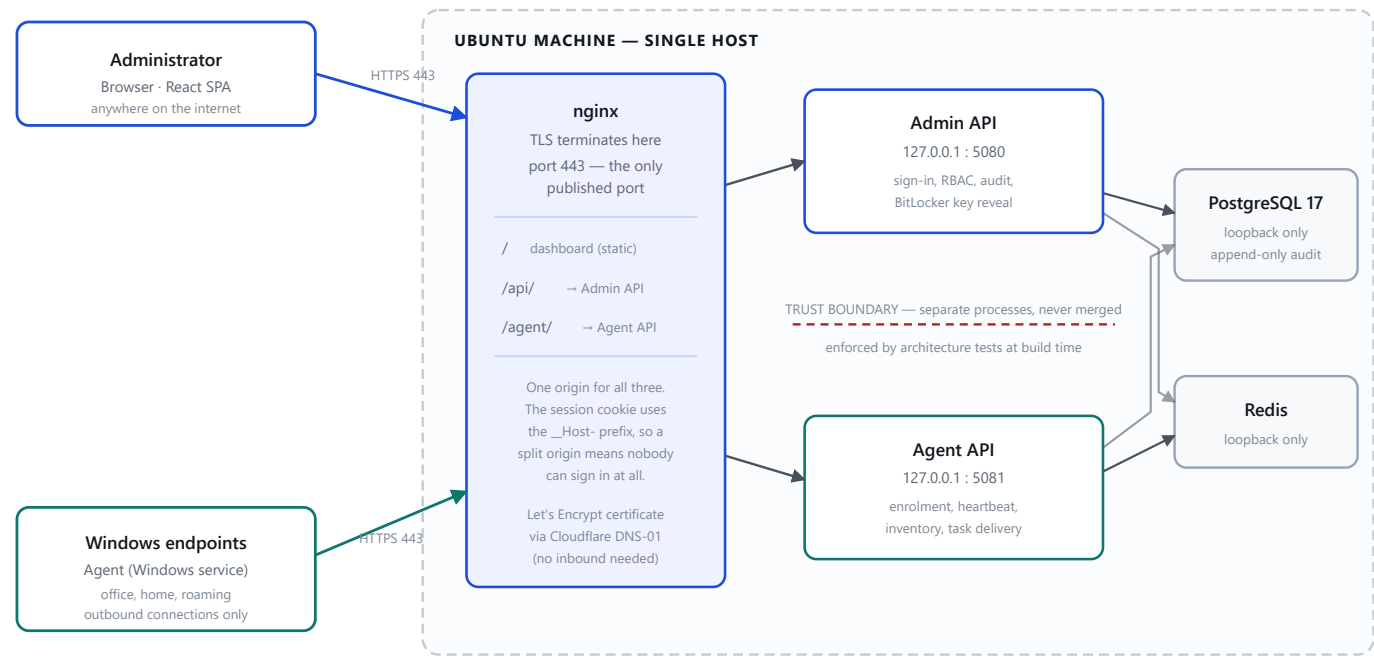


System architecture

Central management for organisation-owned Windows computers. Three .NET processes run under `systemd` behind `nginx` on one Ubuntu machine. Nothing runs in a container.



Every arrow is outbound from the endpoint. No inbound port is ever opened on a managed PC.

THE TWO API HOSTS

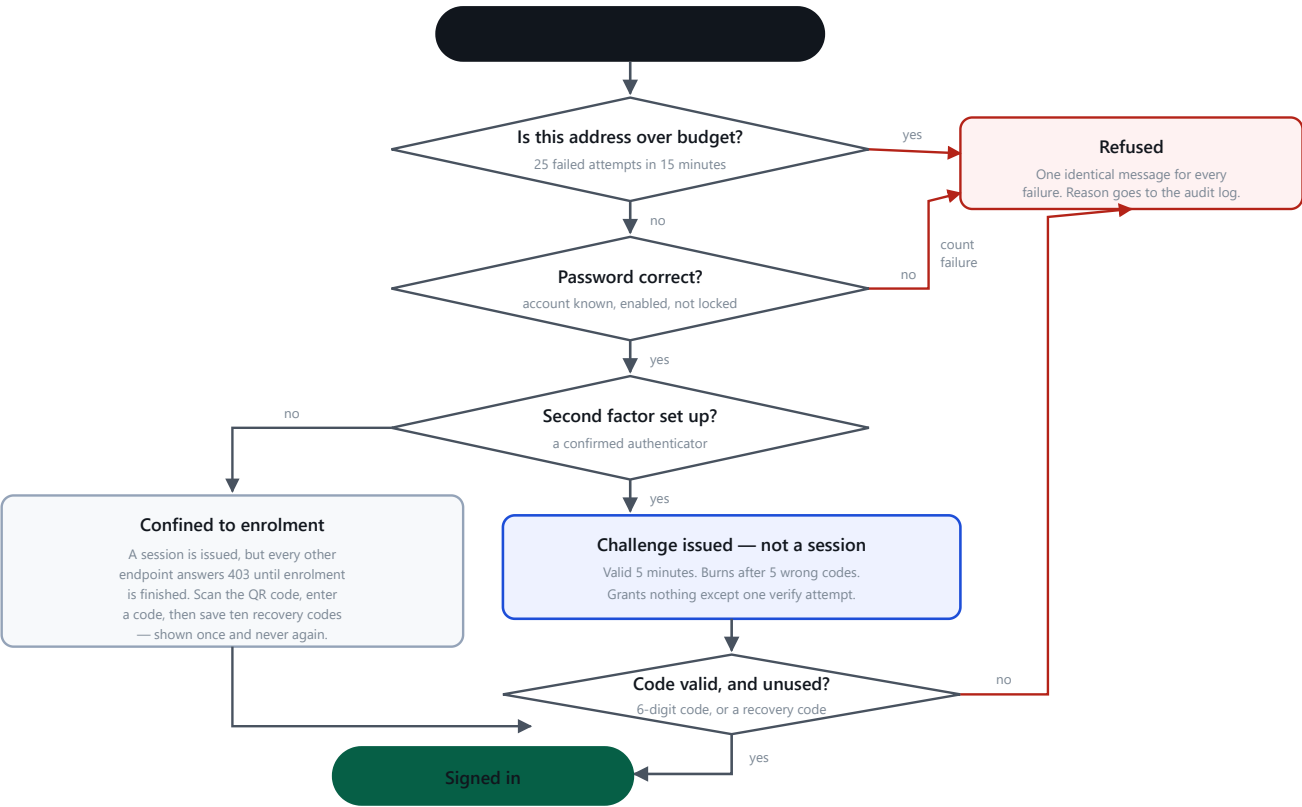
	ADMIN API	AGENT API
Who calls it	A human administrator	An enrolled machine
Authenticates with	Session cookie or bearer token, after password + second factor	Per-device credential issued at enrolment
Can decrypt recovery keys	Yes	No — the key is withheld from this process
CORS	Dashboard origin only	None — machine to machine

Why two processes rather than one. A stolen device credential must never reach an administrative endpoint, and an administrator's session must never be replayable against an agent endpoint. Separation is not a convention here: the build fails if either host references the other.

Publish port 443 only. Both APIs bind to loopback and trust `X-Forwarded-For` from `nginx`. Forwarding 5080 or 5081 straight through would let anyone on the internet forge their own client IP and assert that their request arrived over HTTPS.

Administrator sign-in

Two-factor authentication is mandatory for every administrator and cannot be switched off. A correct password alone never produces a session.



A code works only once, even during the ~90 seconds it stays mathematically valid.

PROTECTING THE ACCOUNT

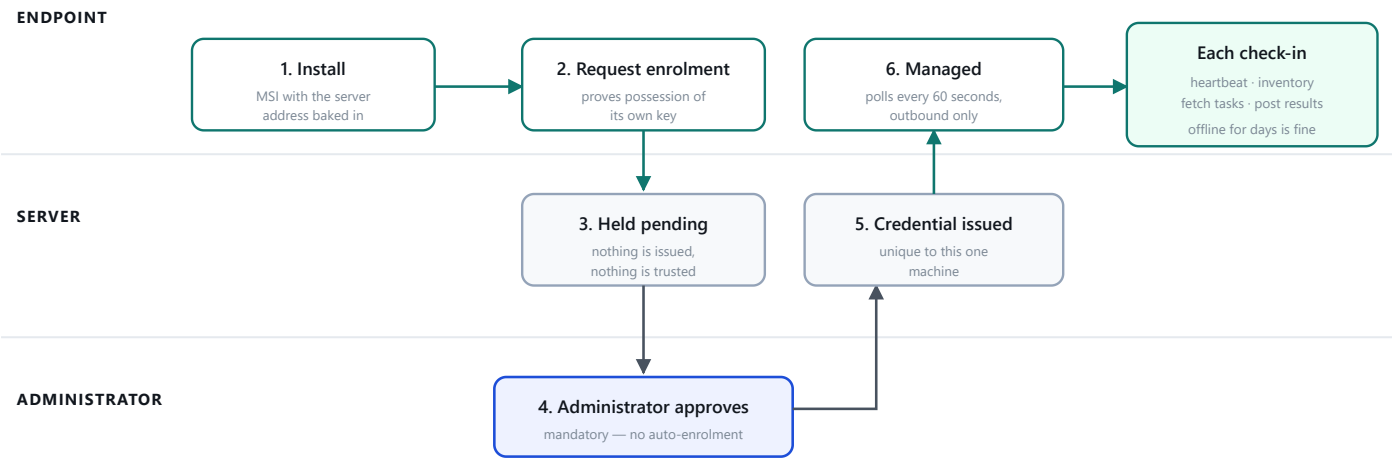
Lockout	5 failures locks for 15 minutes
Forgetting	The count decays after 60 minutes
Session life	12 hours, no sliding renewal
Password floor	12 characters, screened for guessability

PROTECTING AGAINST LOCKOUT ABUSE

An address that keeps failing is refused for 15 minutes, and while refused it **cannot drive any account's lockout counter**. Without that pairing, anyone who knew an administrator's e-mail address — which is simply their username — could keep that person locked out indefinitely at almost no cost.

Agent lifecycle

A managed PC never listens for instructions. It connects outbound, on its own schedule, and receives nothing at all until an administrator has approved it.



Backoff is jittered from 30 seconds to 5 minutes, so a fleet never stampedes a recovering server.

BITLOCKER RECOVERY KEYS

Recovery passwords are the most sensitive data the platform holds, and there are two ways one arrives. Both end up encrypted at rest under a key that **only the Admin API process holds** — the endpoint-facing Agent API is started without it and refuses to run if it is ever given it.

PATH	HOW THE KEY TRAVELS	WHO CAN READ IT IN TRANSIT
Automatic	The agent encrypts it on the endpoint before sending. The Agent API stores something it cannot itself open.	Nobody
Manual	An administrator types it into the console; the server encrypts it on arrival.	Protected by HTTPS alone

Revealing a key is deliberately awkward: it needs the right permission, a re-entered password, and it is rate limited per administrator and per device. Every reveal is recorded.

The audit trail cannot be edited. Audit rows are append-only, enforced by the database itself: the account the application runs as has had UPDATE and DELETE on that table revoked. Even a full compromise of the application cannot quietly rewrite history.

Two keys that cannot be replaced if lost. `RECOVERY_ESCROW_KEY` decrypts every escrowed BitLocker password; `MFA_TOTP_KEY` decrypts every authenticator enrolment. Both live in `/etc/endpoint-platform/secrets.env`. Losing that file does not degrade the platform — it makes recovery keys permanently unreadable and stops every administrator signing in. Back it up wherever the database backups go, and somewhere the platform itself does not depend on.